

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 2
Introduction

Data Structures and Algorithms

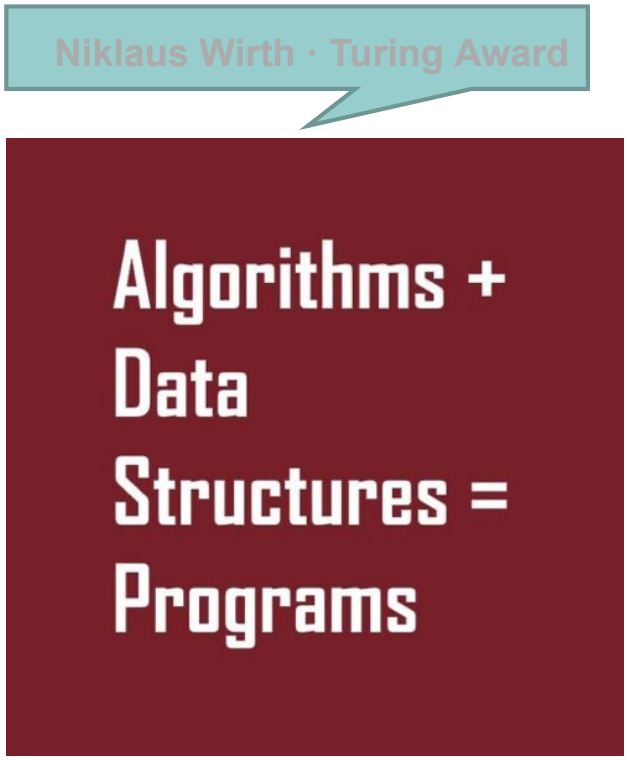
Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Data Structures and Algorithms

- In computer science, a data structure is a particular way of organizing data in a computer so that it can be used efficiently.
- An algorithm is a step-by-step procedure for calculations.
- A program, is a sequence of instructions, written to perform a specified task with a computer.

- ***Algorithms + Data Structures = Programs***
- A 1976 book written by Niklaus Wirth, who won the Turing award in 1984.



Terminologies

- **数据(Data):**是对信息的一种符号表示。在计算机科学中是指所有能输入到计算机中并被计算机程序处理的符号的总称。
- **数据类型(Data Type):** 一个值的集合和定义在这个值集上的一组操作的总称。数据类型主要表明数据的取值范围和操作特性，是具有相同取值范围和可实施同种操作的数据集合的总称。
- **数据元素(Data Element):**能独立、完整地描述问题世界中的实体的最小数据单位称为**数据元素**(也称**记录**)。构成数据元素的不可分割的数据单位称为**数据项**。
- **数据对象(Data Object):**是性质相同的数据元素的集合。
- **数据结构(Data Structure):**是相互之间存在一种或多种特定关系的数据元素的集合。

Terminologies

- Representation
  - Data-Structure=(D，R)
  - D: Data objects
  - R: Relations

- 数据结构是相互间具有某些关系的数据元素的集合。它包括下面三方面的内容：

1）**数据的逻辑结构：**表示数据元素间的关系；

2）**数据的存储结构：**指如何在计算机上存储数据元素及其关系；

3）**数据的运算（集合）：**是数据元素集上所允许的操作

Terminologies

- 数据之间的相互关系称为逻辑结构。可分为四类基本结构：

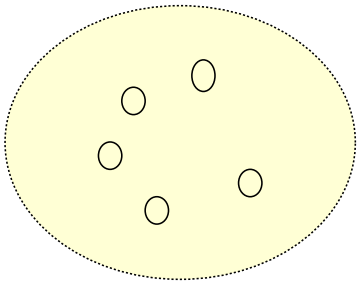
1）**集合（set）结构**：结构中的数据元素除了同属于一种类型外，别无其它关系（松散关系）；

2）**线性（linear）结构**：结构中的数据元素之间存在一对一的关系；

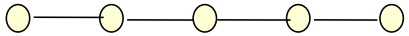
3）**树型（tree）结构**：结构中的数据元素之间存在一对多的关系；

4）**图型结构（graph）或网状结构**：结构中的数据元素之间存在多对多的关系。

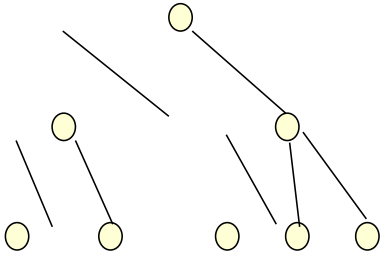
Terminologies



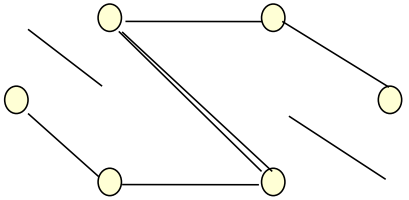
(a) set



(b) linear



(c) tree



(d) graph

- Tree structure and Graph structure are called non-linear structure

An Example

教务处人事简表(表1-1）

职工号	姓名	性别	出生日期	职务	部门
01	万明华	男	52.03.20	处长	
02	赵　宁	男	58.06.14	科长	教学科
03	张　利	女	54.12.07	科长	学籍科
04	赵书芳	女	62.08.05	科长	考务科
05	刘永年	男	49.08.15	科员	教学科
06	王明理	男	65.04.01	科员	学籍科
07	王　敏	女	62.06.28	科员	学籍科
08	张　才	男	57.03.17	科员	学籍科
09	马立仁	男	65.10.12	科员	考务科
10	邢怀常	男	66.07.05	科员	考务科

- Consider the following relations:
  - Ages
  - Posts
  - Departments

List of data structures

- Data type
  - Primitive types: Boolean, Integer, Double, Character, Enum, etc.
  - Composite types: Array, Struct, Union, etc.
  - Abstract data types: Stack, List, Map, Set, String, Queue, etc.
- Linear data structures
  - Arrays
  - Lists
- Trees
  - Binary trees
  - Heap
  - Multiway trees
  - …

Abstract Data Type (ADT)

- An Abstract Data Type (ADT) is a data type that is organized in such a way that the specification on the objects and specification of the operations on the objects are separated from the representation of the objects and the implementation on the operations.
- 用三元组描述如下：

  ADT =（D,S,P）

其中：D是数据对象，S是D上的关系集，P是D的基本操作集。

ADT 抽象数据类型名{

数据对象：〈数据对象的定义〉

数据关系：〈数据关系的定义〉

基本操作：〈基本操作的定义〉

} ADT 抽象数据类型名

Algorithms

- Definition: An algorithm is a finite set of instructions that, if followed, accomplishes a particular task. In addition, all algorithms must satisfy the following criteria:

(1) Input: There are zero or more quantities that are externally supplied.

(2) Output : At least one quantity is produced.

(3) Definiteness（确定性）: Each instruction is clear and unambiguous.

(4) Finiteness（有穷性）: If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after finite number of steps.

(5) Effectiveness（有效性）: Every instruction must be basic enough to be carried out, in principle, by a person using only pencil and paper.

Algorithms

- 评价一个好的算法有以下几个标准：

(1)正确性(Correctness)：算法应满足具体问题的需求。

(2)可读性(Readability)：算法应该好读。以有利于阅读者对程序的理解。

(3)健状性(Robustness)：算法应具有容错处理。当输入非法数据时，算法应对其作出反应，而不是产生莫名其妙的输出结果。

(4)效率与存储量需求：效率指的是算法执行的时间；存储量需求指算法执行过程中所需要的最大存储空间。一般，这两者与问题的规模有关。

Algorithms

- 算法的描述方法可以归纳为以下几种：

  (1) 自然语言，如中文、英文；

  (2) 图形，如N-S图、流程图，图的描述与算法语言的描述对应；

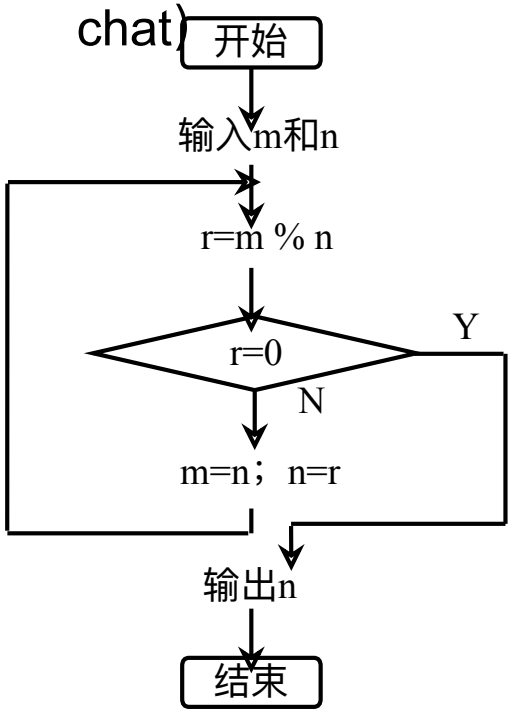
  (3) 算法语言，即计算机语言、程序设计语言、伪代码；

  (4) 形式语言，用数学的方法，可以避免自然语言的二义性。

- 用各种算法描述方法所描述的同一算法，该算法的功用是一样的，允许在算法的描述和实现方法上有所不同。

Example: Euclidean algorithm

- 自然语言（Natural language）
  - ① 输入m 和n；
  - ② 求m除以n的余数r；
  - ③ 若r等于0，则n为最大公约数，算法结束；
    否则执行第④步；
  - ④ 将n的值放在m中，将r的值放在n中；
  - ⑤ 重新执行第②步。
- 伪代码（Pseudo code）
  - 1. r = m % n;
  - 2. iteration until r = 0
  - 2.1 m = n;
  - 2.2 n = r;
- 流程图（Flow chat）



Evaluating algorithms

- Correctness
  1. Theoretical correctness
  2. Numerical stability
- Efficiency
  1. Time
  2. Space
- An algorithm is efficient if it uses as few resources as possible. In many situations there is a trade-off between time and space, in that an algorithm can be made faster if it uses more space or smaller if it takes longer.
- Although a thorough analysis of an algorithm should consider both time and space, time is considered more important.

Measuring time

- How should we measure the time taken by an algorithm?
- We can do it experimentally by measuring the number of seconds it takes for a program to run — this is often called benchmarking and is often seen in popular magazines. This can be useful, but depends on many factors:
  - The machine on which it is running.
  - The language in which it is written.
  - The skill of the programmer.
  - The instance on which the program is being run, both in terms of size and which particular instance it is.
- So it is not an independent measure of the algorithm, but rather a measure of the implementation, the machine and the instance.

Complexity

- The complexity of an algorithm is a “device-independent” measure of how much time it consumes. Rather than expressing the time consumed in seconds, we attempt to count how many “elementary operations” the algorithm performs when presented with instances of different sizes.
- The result is expressed as a function, giving the number of operations in terms of the size of the instance. This measure is not as precise as a benchmark, but much more useful for answering the kind of questions that commonly arise:
  - I want to solve a problem twice as big. How long will that take me?
  - We can afford to buy a machine twice as fast? What size of problem can we solve in the same time?
- The answers to questions like this depend on the complexity of the algorithm.

Asymptotic notations

- Big-O notation: It defines an asymptotic upper bound for a function *f(n)*.

  Definition: A function *f(n)* is said to be *O(g(n))* if there are constants *c* and *n0* such that

  $f(n) \le cg(n), \forall n \ge n0$.

- Big-Omega notation (□): It defines an asymptotic lower bound for a function *f(n)*.
- Big-Theta notation: It defines an asymptotic upper and lower bound for a function *f(n)*.

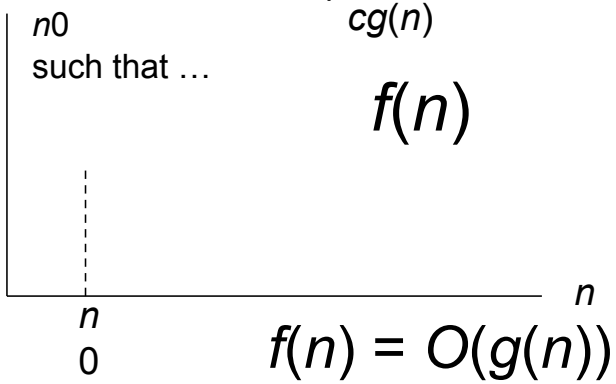
  Definition: A function *f(n)* is said to be □*(g(n))* if there are constants *c1, c2* and *n0* such that

  $0 \le c1g(n) \le f(n) \le c2g(n), \forall n \ge n0$.

- If we say that *f(n)* = □*(n2)* then we are implying that *f(n)* is approximately proportional to *n2* for large values of *n*.

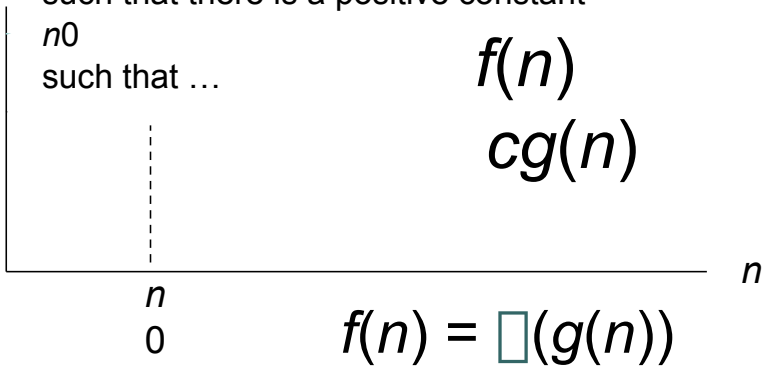
Asymptotic notations

There exist positive constants *c*
such that there is a positive constant
*n*0
such that …



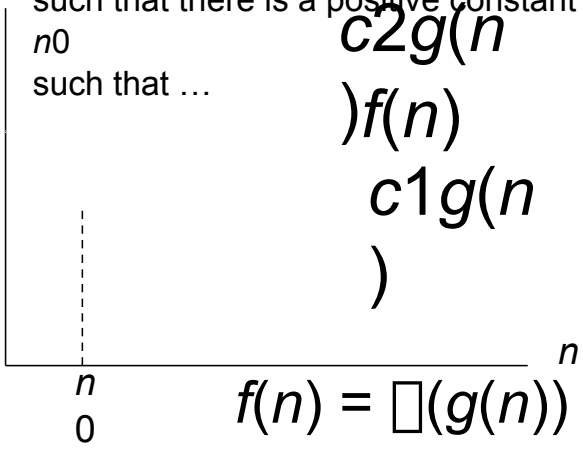
$f(n) = O(g(n))$

There exist positive constants *c*
such that there is a positive constant
*n*0
such that …



$f(n) = \Omega(g(n))$

There exist positive constants *c*1 and
*c*2
such that there is a positive constant
*n*0
such that …



$f(n) = \Omega(g(n))$

Asymptotics

> DEFINITION ***Asymptotics*** is the study of functions of a parameter n, as n becomes larger and larger without bound.

- Suppose the function *f(n)* measures the amount of work done by an algorithm on a problem of size *n*. We compare *f(n),* for large values of *n*, with some well-known function *g(n)* whose behavior we already understand. Common choices for *g(n):*
  - *g(n) = 1* *Constant function*
  - *g(n) = log n* *Logarithmic function*
  - *g(n) = n* *Linear function*
  - *g(n) = n2* n^2 *Quadratic function*
  - *g(n) = n3* n^3 *Cubic function*
  - *g(n) = 2n* *Exponential function*

Asymptotics



DEFINITION If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ then:
$f(n)$ has ***strictly smaller order of magnitude*** than $g(n)$.

If $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ is finite and nonzero then:
$f(n)$ has ***the same order of magnitude*** as $g(n)$.

If $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$ then:
$f(n)$ has ***strictly greater order of magnitude*** than $g(n)$.

DEFINITION $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$ means $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$.

Asymptotic notations

Notation: $f(n)$ *is*	*Meaning:* *Order of* f *compared to* g *is*		*Value of* $\lim_{n\to\infty}\big(f(n)/g(n)\big)$
$o\big(g(n)\big)$	$<$	*strictly smaller*	0
$O\big(g(n)\big)$	$\leq$	*smaller or equal*	*finite*
$\Theta\big(g(n)\big)$	$=$	*equal*	*nonzero finite*
$\Omega\big(g(n)\big)$	$\geq$	*larger or equal*	*nonzero*

L'Hôpital's Rule Suppose that:

- $f(x)$ and $g(x)$ are differentiable functions for all sufficiently large x, with derivatives $f'(x)$ and $g'(x)$, respectively.
- $\lim_{x\to\infty} f(x) = \infty$ and $\lim_{x\to\infty} g(x) = \infty$.
- $\lim_{x\to\infty} \dfrac{f'(x)}{g'(x)}$ exists.

Then $\lim_{x\to\infty} \dfrac{f(x)}{g(x)}$ exists and $\lim_{x\to\infty} \dfrac{f(x)}{g(x)} = \lim_{x\to\infty} \dfrac{f'(x)}{g'(x)}$.

Order of growth

- Asymptotic notations: □, *O*, □, *o*, □.
- The constant coefficient(s) are ignored.
- Lower order item(s) are ignored, just keep the highest order item.
- The rate of growth, or the order of growth, possesses the highest significance.

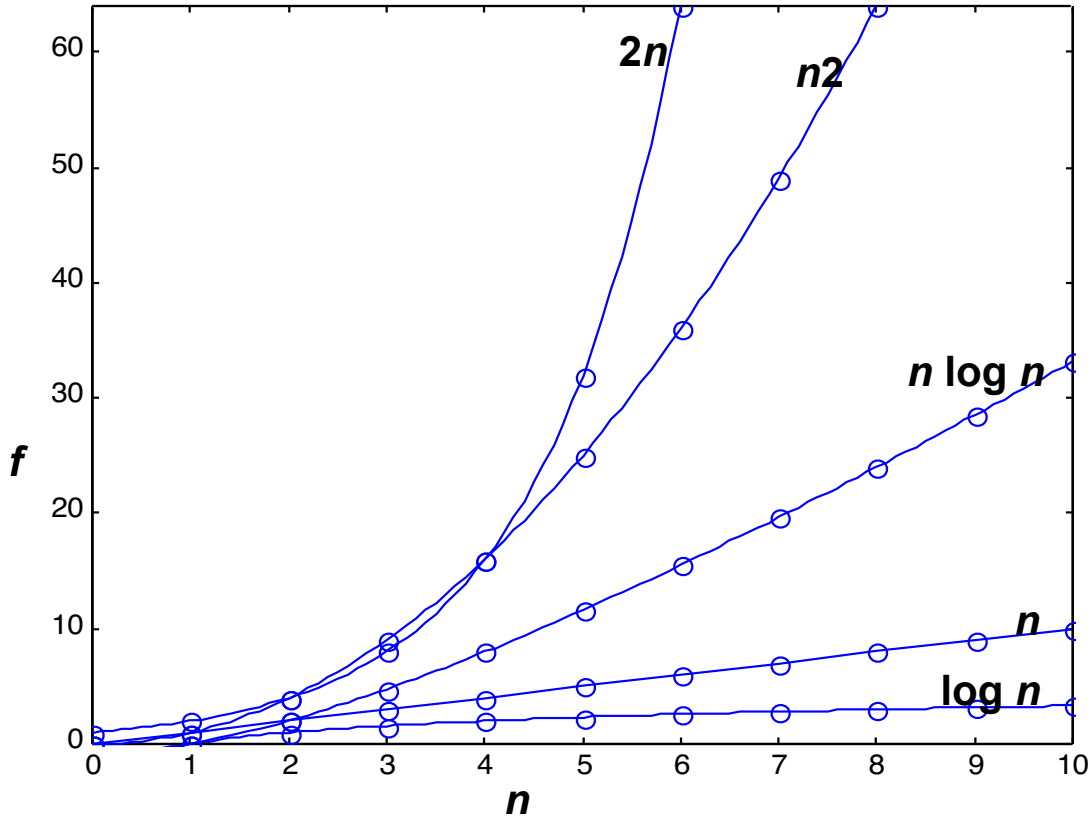
- The sequential search runs in □(*n*).
- *f*(*n*)= □(*g*(*n*)) actually means： *f*(*n*) □ □(*g*(*n*))。

- Typical order of growth: O(1), O(log *n*), O(□*n*), O(*n*), O(*n*log *n*), O(*n*2), O(*n*3), O(2*n*), O(*n*!).

Arithmetic operation

- $O(f(n))+O(g(n)) = O(\max\{f(n),g(n)\})$;
- $O(f(n))+O(g(n)) = O(f(n)+g(n))$;
- $O(f(n))*O(g(n)) = O(f(n)*g(n))$;
- $O(cf(n)) = O(f(n))$;
- $g(n)= O(f(n))$ □ $O(f(n))+O(g(n)) = O(f(n))$ 。

Order of growth



Exercise

- List the following functions by increasing asymptotic growth rate.

$$\lg n \qquad 1.1^n \qquad n\lg n \qquad n(\lg n)^2 \qquad 3\lg n \qquad 2^5 \qquad n^{34}$$

Thank you!

